

41052 - Advanced Algorithms

Programming Assignment 1 - Fast Fourier Transform

Tung Nguyen - 24666048

Contents

1	A little history	2
2	Relevant background knowledge	2
2.1	Discrete Fourier Transform	2
2.2	Fast Fourier Transform	3
2.2.1	Iterative Radix-2 FFT	3
2.2.2	Recursive Radix-2 FFT	6
2.2.3	Optimising with Precomputed Twiddle	8
2.2.4	Mixed Radix Fast Fourier Transform	9
3	Code Implementation	11
3.1	Architecture & Repository Navigation	11
3.2	Data Representation & Hardware Cache Alignment	12
3.3	Practical changes	12
3.3.1	Reusing FFT for IFFT	12
3.3.2	Code wrappers	13
3.3.3	Smart finding factor function	13
3.3.4	Catching error	14
4	The Experiments and Empirical Study	14
4.1	Benchmarking Environment & Measurement Rigour	14
4.2	Experiment 1: Asymptotic Scaling & Crossover Point	15
4.3	Experiment 2: Call-Stack Overhead & In-Place Bit-Reversal	16
4.4	Experiment 3: On-the-fly Math vs. Precomputed Twiddles	18
4.5	Experiment 4: The Hardware <i>Cache Cliff</i> Effect	20
4.6	Experiment 5: Numerical Precision Drift	22
4.7	Experiment 6: Mixed-Radix Factorisation & Prime Degradation	24
5	What I learned	26
6	AI usage Declaration	27

1 A little history

The Discrete Fourier Transform wasn't invented by Joseph Fourier. It actually predated Fourier's work on the continuous Fourier Transform (formally published in 1822)! In 1759, Alexis Clairaut published an early discrete trigonometric formula while calculating celestial orbits [?]. In 1805, Carl Friedrich Gauss refined this technique to track the paths of asteroids, effectively devising both the discrete Fourier formula and an early Fast Fourier Transform (FFT) [?]. However, his work remained largely overlooked until long after his death because it was recorded in unpublished Latin notes with unconventional notation. Since this formula handles discrete values with methods like the Fourier transform, later on it was named as the Discrete Fourier Transform.

The modern breakthrough occurred during the Cold War. As the United States and the Soviet Union negotiated nuclear test bans, verifying compliance required distinguishing clandestine underground nuclear tests from natural earthquakes [?]. Processing the flood of discrete seismic sensor data was computationally impractical using standard methods, which scaled at $\mathcal{O}(N^2)$.

Physicist and defence adviser Richard Garwin recognised this bottleneck and raised it with statistician John Tukey from Princeton and Bell Labs. Tukey proposed a divide-and-conquer approach to accelerate the computation and worked with IBM mathematician James Cooley, who implemented and programmed the algorithm. Published in 1965, the Cooley–Tukey FFT slashed the computational cost to $\mathcal{O}(N \log_2 N)$ [?] (fundamentally, Gauss invented the same algorithm but his work went unnoticed). But by that time, major countries has already created their nuclear devices and went ahead with their own testing.

Nowadays, the FFT has revolutionised digital signal processing, telecommunications and numerical computing worldwide, thus becoming one of the most important algorithm in our lives.

2 Relevant background knowledge

This project will be implementing and studying five relevant algorithms.

1. Standard Discrete Fourier Transform
2. Iterative Radix-2 Fast Fourier Transform (FFT)
3. Recursive Radix-2 FFT
4. Iterative Radix-2 with Precomputed Twiddle
5. Mixed Radix FFT

This section will discuss the theoretical background of the above mentioned methods, mainly how to implement the algorithms and their complexity.

2.1 Discrete Fourier Transform

The Discrete Fourier Transform (DFT) is a function that receives an array of complex numbers $x = (x_0, x_1, \dots, x_{N-1})$ and returns a transformed frequency array $X = (X_0, X_1, \dots, X_{N-1})$ [?]. Formally, the DFT is defined as:

$$X_k = \sum_{j=0}^{N-1} x_j \cdot W, \quad (1)$$

with $W = \cos(\theta) - i \sin(\theta)$, and $\theta = \frac{2\pi k j}{N}$. The DFT can be defined as an algorithm.

Algorithm 1 Discrete Fourier Transform

Input: Array $x[0 \dots N-1]$ of complex numbers, length N

Output: Transformed array $X[0 \dots N-1]$

```
1: procedure DFT( $x, N$ )
2:   Allocate array  $X[0 \dots N-1]$  initialised to 0
3:   for  $k \leftarrow 0$  to  $N-1$  do
4:     for  $j \leftarrow 0$  to  $N-1$  do
5:        $\theta \leftarrow \frac{2\pi}{N} \cdot k \cdot j$ 
6:        $W \leftarrow \cos \theta - i \sin \theta$ 
7:        $X[k] \leftarrow X[k] + x[j] \cdot W$ 
8:   return  $X$ 
```

The W is known as the twiddle factor

The Inverse Discrete Fourier Transform (IDFT) is used for reversing a transformed array to its original array, essentially undo the DFT. It can also be described as an algorithm.

Algorithm 2 Inverse Discrete Fourier Transform

Input: Transformed array $X = [X[0], X[1], \dots, X[N-1]]$ of size N

Output: Original array $x = [x[0], x[1], \dots, x[N-1]]$

```

1: procedure IDFT( $x, N$ )
2:   Allocate array  $x[0 \dots N-1]$  initialised to 0
3:   for  $j \leftarrow 0$  to  $N-1$  do
4:     for  $k \leftarrow 0$  to  $N-1$  do
5:        $\theta \leftarrow \frac{2\pi}{N} \cdot k \cdot j$ 
6:        $W \leftarrow \cos(\theta) + i \sin(\theta)$ 
7:        $x[j] \leftarrow x[j] + X[k] \cdot W$ 
8:    $x[j] \leftarrow \frac{1}{N} \cdot x[j]$ 
9:   return  $x$ 

```

Once again, due to the nested for loops (line 2 and 3), the IDFT has a complexity of $\mathcal{O}(N^2)$. Moreover, it is true that

$$\text{IDFT}(x) = \frac{1}{N} \cdot \text{conj}[\text{DFT}(\text{conj}(x))]. \quad (2)$$

In this property, $\text{conj}()$ is a function that turn a complex number into its complex conjugate (flipping the operator of the imaginary part). This operation and the normalisation over N (line 8) is looping over N input of the array, hence it's complexity is $\mathcal{O}(N)$ and is bounded by the above mentioned $\mathcal{O}(N^2)$. Simply speaking, for all of the mentioned algorithm, the IFFT is just the FFT with different factors involved due to the $\text{conj}()$.

2.2 Fast Fourier Transform

The Fast Fourier Transform is a class of algorithms that can compute the DFT and IDFT faster than $\mathcal{O}(N^2)$. In addition, as mentioned in equation 2, when computing for IDFT, a new complexity of $\mathcal{O}(N)$ is added. Since FFT essentially compute DFT, Inverse Fast Fourier Transform (IFFT) can be used to compute IDFT. And for all of the following algorithms, the $\text{conj}()$ complexity is also bounded.

2.2.1 Iterative Radix-2 FFT

The iterative version runs with the help of the Bit Reversal Permutation (BRP) algorithm [?]. The BRP essentially loop over an array, for each index it flips the binary (e.g. $001 \mapsto 100, 110 \mapsto 011$), and move the current value to its new index.

Algorithm 3 Bit-Reversal Permutation

Input: Array $A[0 \dots N-1]$, length $N = 2^\ell$

Output: Array A permuted in-place into leaf order

```

1: procedure BIT-REVERSE-PERMUTATION( $A, N$ )
2:    $\ell \leftarrow \log_2 N$ 
3:   for  $i \leftarrow 0$  to  $N-1$  do
4:      $rev \leftarrow 0$ 
5:      $temp \leftarrow i$ 
6:     for  $b \leftarrow 0$  to  $\ell-1$  do
7:        $bit \leftarrow temp \bmod 2$                                      # Extract binary split parity
8:        $rev \leftarrow 2 \cdot rev + bit$                                # Shift accumulated bits left and append
9:        $temp \leftarrow \lfloor temp/2 \rfloor$                                # Advance to next binary digit
10:    if  $i < rev$  then
11:       $swap(A[i], A[rev])$ 

```

Table 1: Bit-reversal permutation mapping for $N = 8$ ($\ell = 3$ bits).

Original Index (i)	Binary ($b_2b_1b_0$)	Reversed ($b_0b_1b_2$)	Permuted Index ($\text{rev}(i)$)	Action
0	000	000	0	Identity
1	001	100	4	Swap (1, 4)
2	010	010	2	Identity
3	011	110	6	Swap (3, 6)
4	100	001	1	(Handled at $i = 1$)
5	101	101	5	Identity
6	110	011	3	(Handled at $i = 3$)
7	111	111	7	Identity

Algorithm 4 Iterative Radix-2 FFT

Input: Array $x[0 \dots N - 1]$ of complex numbers, length $N = 2^\ell$

Output: Array $x[0 \dots N - 1]$ transformed in-place

```

1: procedure FFT( $x, N$ )
2:   BIT-REVERSE-PERMUTATION( $x, N$ )
3:   for  $s \leftarrow 1$  to  $\log_2 N$  do
4:      $m \leftarrow 2^s$ 
5:      $\theta \leftarrow \frac{2\pi}{m}$ 
6:      $W_m \leftarrow \cos(\theta) - i \sin(\theta)$ 
7:     for  $k \leftarrow 0$  to  $N - 1$  step  $m$  do
8:        $W \leftarrow 1$ 
9:       for  $j \leftarrow 0$  to  $m/2 - 1$  do
10:         $t \leftarrow W \cdot x[k + j + m/2]$ 
11:         $u \leftarrow x[k + j]$ 
12:         $x[k + j] \leftarrow u + t$ 
13:         $x[k + j + m/2] \leftarrow u - t$ 
14:         $W \leftarrow W \cdot W_m$ 
15:   return  $x$ 

```

Equivalence

The Radix-2 FFT operates based on the divide-and-conquer strategy, by keep separating the even and odd indices. For example,

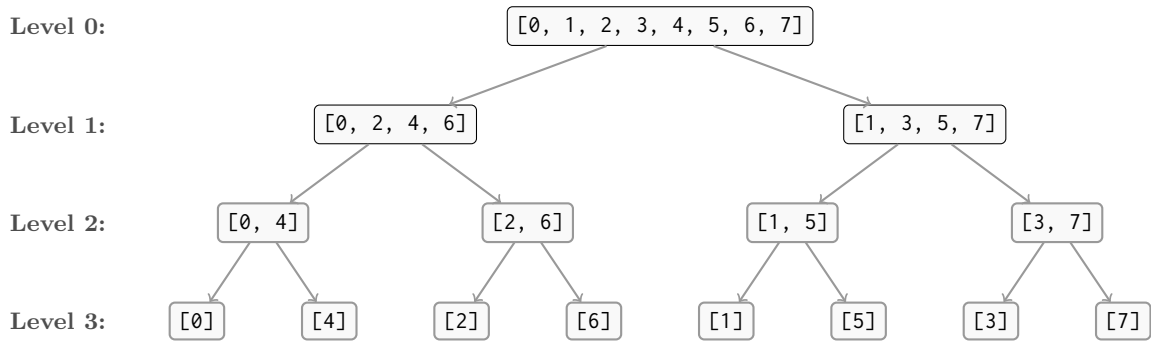


Figure 1: Divide-and-conquer decomposition of an $N = 8$ sequence into bit-reversed leaf order.

The Iterative Radix-2 FFT essentially apply the twiddles from the bottom up. With the BRP, the array is laid out in the form of leaf nodes of the recursion tree sequentially in memory so we can build upward with simple loops. From the example in Table 1, we can have

$$\{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \mapsto \{x_0, x_4, x_2, x_6, x_1, x_5, x_3, x_7\}.$$

Notice that $m = 2^s$ with s starting from 1. This is essentially level 3 from Figure 1. Visually, at $s = 1, m = 2$, we have

$$(x_0 \oplus x_4), (x_2 \oplus x_6), (x_1 \oplus x_5), (x_3 \oplus x_7)$$

We know that the first pair is using indices 0 and 1, the second is using 2 and 3, and so on. So each pair is using an even and an odd index. In each pair, the first element acts as the *even* sub-result and the second acts as the *odd* sub-result. Applying a single black box operation to each pair combines these four adjacent pairs into four independent 2-point DFT solutions, fully updated in-place.

Next, at $s = 2$ ($m = 4$), the problem size doubles (level 2). The algorithm *merges* adjacent pairs of these 2-point solutions into 4-point solutions:

$$([x_0, x_4] \oplus [x_2, x_6]), ([x_1, x_5] \oplus [x_3, x_7])$$

Here, the first pair x_0, x_4 represents the even-indexed frequency components, while x_2, x_6 represents the odd ones. The loop runs two butterflies ($j = 0, 1$) across an offset of $m/2 = 2$ slots in memory, updating each block into a valid 4-point DFT.

Finally, at level 1, at $s = 3$ ($m = 8$), the algorithm performs the last merge step:

$$([x_0, x_4, x_2, x_6] \oplus [x_1, x_5, x_3, x_7])$$

The left block contains all even values of the original array, and the right block contains all odd values. Running four butterflies ($j = 0, 1, 2, 3$) across an offset of $m/2 = 4$ combines them into the final, complete 8-point DFT array.

Since each stage simply merges smaller, already-transformed sub-arrays into larger ones (same as bottom-up Merge Sort). The loops mathematically evaluate the exact same divide-and-conquer tree as the recursive DFT definition.

Moving on, on line 12 and 13, we can see that they are $u + t$ and $u - t$, respectively. To understand this, we need to work on the original DFT in equation 1.

$$\begin{aligned} X_k &= \sum_{j=0}^{N-1} x_j \cdot W \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot W_N^{2mk} + \sum_{m=0}^{N/2-1} x_{2m+1} \cdot W_N^{(2m+1)k} \quad (\text{Splitting into even and odd halves}) \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot (W_N^2)^{mk} + \sum_{m=0}^{N/2-1} x_{2m+1} \cdot W_N^k \cdot W_{N/2}^{mk} \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot W_{N/2}^{mk} + W_N^k \cdot \sum_{m=0}^{N/2-1} x_{2m+1} \cdot W_{N/2}^{mk} \\ &= E_k + W_N^k \cdot O_k. \end{aligned} \tag{3}$$

We can see that $u \equiv E_k$ since $k + j$ is the first of the pair, which is even. And, $t \equiv W_N^k \cdot O_k$ since $k + j + m/2$ is the second of the pair, which is odd. Thus, based on line 13, we have

$$X_{k+m/2} = E_{k+m/2} - W_N^k \cdot O_k.$$

We then have

$$E_{k+m/2} = \sum_{m=0}^{N/2-1} x_{2m} \cdot W_{N/2}^{m(k+m/2)}.$$

Specifically,

$$W_{N/2}^{m(k+m/2)} = W_{N/2}^{mk} \cdot \left(W_{N/2}^{m/2}\right)^m = W_{N/2}^{mk} \cdot 1^m = W_{N/2}^{mk}.$$

Note that after the iteration, which is when this FFT is completed, $m = 2^{\log_2 N} = N$. Thus,

$$W_{N/2}^{m/2} = W_{N/2}^{N/2} = \cos\left(-2\pi \cdot \frac{N}{2}\right) - i \sin\left(-2\pi \cdot \frac{N}{2}\right) = 1.$$

Thus, $E_k = E_{k+m/2}$. Same holds true for $O_k = O_{k+m/2}$. As for W_m^k , we have

$$W_m^{k+m/2} = W_m^k \cdot W_m^{m/2} = W_m^k \cdot \left[\cos\left(2\pi \cdot \frac{-m/2}{m}\right) - i \sin\left(-2\pi \cdot \frac{m/2}{m}\right)\right] = W_m^k \cdot -1 = -W_m^k.$$

Therefore, line 13 is valid.

Complexity

Looking deeper into the Iterative version of the Radix-2 FFT, the first part of the algorithm is spent running the BRP algorithm, which first cost $\mathcal{O}(N)$ looping over the entire array. And a nested for loop inside that cost $\mathcal{O}(\log_2 N)$. In total, the BRP algorithm cost $\mathcal{O}(N \log_2 N)$. Next, the algorithm loops (line 4) over $\log_2 N$ times, followed by a nested loop (line 7) of $\frac{N}{m}$ times (indicated by the **step** m), followed by another nested loop (line 9) of $\frac{m}{2}$ times. In total, this FFT algorithm cost $\mathcal{O}(N \log_2 N + \log_2 N \cdot \frac{N}{m} \cdot \frac{m}{2}) = \mathcal{O}(N \log_2 N)$. Also, since this is in-place, the space complexity only needs to account for the original input, which is $\mathcal{O}(N)$.

Algorithm 5 Iterative Radix-2 IFFT

Input: Transformed array $X[0 \dots N-1]$ of complex numbers, length $N = 2^\ell$

Output: Original array $x[0 \dots N-1]$

```
1: procedure IFFT( $x, N$ )
2:    $m \leftarrow \log_2 N$ 
3:   BIT-REVERSE-PERMUTATION( $x, N$ )
4:   for  $s \leftarrow 1$  to  $\log_2 N$  do
5:      $m \leftarrow 2^s$ 
6:      $\theta \leftarrow \frac{2\pi}{m}$ 
7:      $W_m \leftarrow \cos(\theta) + i \sin(\theta)$ 
8:     for  $k \leftarrow 0$  to  $N-1$  step  $m$  do
9:        $W \leftarrow 1$ 
10:      for  $j \leftarrow 0$  to  $m/2-1$  do
11:         $t \leftarrow W \cdot x[k+j+m/2]$ 
12:         $u \leftarrow x[k+j]$ 
13:         $x[k+j] \leftarrow u+t$ 
14:         $x[k+j+m/2] \leftarrow u-t$ 
15:         $W \leftarrow W \cdot W_m$ 
16:      for  $i \leftarrow 0$  to  $N-1$  do
17:         $x[i] \leftarrow \frac{x[i]}{N}$ 
18:  return  $x$ 
```

2.2.2 Recursive Radix-2 FFT

This is simply a recursive version of the above FFT, so the mathematical derivation is the same.

Algorithm 6 Recursive Radix-2 FFT

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^\ell$

Output: Transformed array $X[0 \dots N-1]$

```
1: procedure FFT( $x, N$ )
2:   if  $N = 1$  then
3:     return  $x$ 
4:   Allocate array  $x_{\text{even}}$  of length  $N/2$ 
5:   Allocate array  $x_{\text{odd}}$  of length  $N/2$ 
6:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
7:      $x_{\text{even}}[k] \leftarrow x[2k]$ 
8:      $x_{\text{odd}}[k] \leftarrow x[2k + 1]$ 
9:    $E \leftarrow \text{FFT}(x_{\text{even}}, N/2)$ 
10:   $O \leftarrow \text{FFT}(x_{\text{odd}}, N/2)$ 
11:  Allocate array  $X$  of length  $N$ 
12:  for  $k \leftarrow 0$  to  $N/2 - 1$  do
13:     $\theta \leftarrow \frac{2\pi}{N} \cdot k$ 
14:     $W \leftarrow \cos(\theta) - i \sin(\theta)$ 
15:     $t \leftarrow W \cdot O[k]$ 
16:     $X[k] \leftarrow E[k] + t$ 
17:     $X[k + N/2] \leftarrow E[k] - t$ 
18:  return  $X$ 
```

Equivalence

The recursive Radix-2 FFT computes the exact same result as the DFT by framing it as a top-down divide-and-conquer problem (level 1 to 3). Instead of evaluating all frequency components at once, it interleaves the input signal into two smaller sequences containing only the even-indexed and odd-indexed samples. Then, it recursively compute the sub-DFTs for each half until reaching the trivial base case of single-element arrays.

Since shifting a sequence in time corresponds to a phase rotation in frequency, the odd sub-transform is multiplied by a twiddle factor to account for its one-sample time offset. Finally, it exploit the circular symmetry of frequencies the $u - t$ proof above). The algorithm reconstructs the full-length DFT by simultaneously computing the first half of the array through addition and the second half through subtraction, precisely reproducing the mathematical sum of the full N -point DFT.

Complexity

The Recursive version of the Radix-2 FFT first copy into two arrays x_{even} and x_{odd} (line 6 to 8), with each costing $\mathcal{O}\left(\frac{N}{2}\right)$. Then, it recursively called with inputs of size $\frac{N}{2}$ two times (line 9 and 10), each time costing $T\left(\frac{N}{2}\right)$ (since it's recursion, we cannot use Big $\mathcal{O}$ right away). Finally, the algorithm loop over $\frac{N}{2}$ times (line 12). Thus, the entire algorithm cost

$$T(N) = 2\mathcal{O}\left(\frac{N}{2}\right) + 2T\left(\frac{N}{2}\right) + \mathcal{O}\left(\frac{N}{2}\right) = \mathcal{O}(N) + 2T\left(\frac{N}{2}\right).$$

According to the Master Theorem [?], we have

$$T(N) = aT\left(\frac{N}{b}\right) + f(N).$$

It is clear that $f(n) = \mathcal{O}(N)$. Moreover, $a = 2, b = 2 \Rightarrow N^{\log_2 2} = N$. This matches with the case 2 of the Master Theorem. Therefore,

$$T(N) = \mathcal{O}(N \log_2 N).$$

In terms of space complexity, due to the allocation of E and O , both are $N/2$ in length, and X of size N , the space complexity is $\mathcal{O}(N \log_2 N)$.

Algorithm 7 Recursive Radix-2 IFFT

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^\ell$ **Output:** Transformed array $X[0 \dots N-1]$

```
1: procedure FFT( $x, N$ )
2:   if  $N = 1$  then
3:     return  $x$ 
4:   Allocate array  $x_{\text{even}}$  of length  $N/2$ 
5:   Allocate array  $x_{\text{odd}}$  of length  $N/2$ 
6:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
7:      $x_{\text{even}}[k] \leftarrow x[2k]$ 
8:      $x_{\text{odd}}[k] \leftarrow x[2k+1]$ 
9:    $E \leftarrow \text{FFT}(x_{\text{even}}, N/2)$                                 # Sub-transform of even elements
10:   $O \leftarrow \text{FFT}(x_{\text{odd}}, N/2)$                                 # Sub-transform of odd elements
11:  Allocate array  $X$  of length  $N$ 
12:  for  $k \leftarrow 0$  to  $N/2 - 1$  do
13:     $\theta \leftarrow \frac{2\pi}{N} \cdot k$ 
14:     $W \leftarrow \cos(\theta) + i \sin(\theta)$ 
15:     $t \leftarrow W \cdot O[k]$ 
16:     $X[k] \leftarrow E[k] + t$ 
17:     $X[k + N/2] \leftarrow E[k] - t$ 
18:  return  $X$ 
```

2.2.3 Optimising with Precomputed Twiddle

On paper, a small optimisation can be done by pre-computing the twiddles factor (W) first [?]. Since a major part of the FFT is to compute the twiddles, naturally having not to deal with them in the actual algorithm is faster.

Algorithm 8 Iterative Radix-2 FFT with Precomputed Twiddle Table

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^\ell$, precomputed table $W[0 \dots N/2 - 1]$ where $W[k] = e^{-i\frac{2\pi}{N}k}$ **Output:** Transformed array $X[0 \dots N-1]$

```
1: procedure PRECOMPUTE-TWIDDLES( $N$ )
2:   Allocate table  $W$  of size  $N/2$ 
3:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
4:      $\theta \leftarrow \frac{2\pi}{N} \cdot k$ 
5:      $W[k] \leftarrow \cos(\theta) - i \sin(\theta)$ 
6:   return  $W$ 

7: procedure IFFT( $x, N, W$ )
8:   BIT-REVERSE-PERMUTATION( $X, N$ )
9:   for  $s \leftarrow 1$  to  $\log_2 N$  do                                # Stage loop
10:     $m \leftarrow 2^s$                                               # Sub-transform size
11:     $\text{step} \leftarrow N/m$                                           # Stride to sample twiddles from global table
12:    for  $k \leftarrow 0$  to  $N - 1$  step  $m$  do                        # Block loop
13:      for  $j \leftarrow 0$  to  $m/2 - 1$  do                            # Butterfly loop
14:         $W \leftarrow W[j \cdot \text{step}]$                             #  $\mathcal{O}(1)$  table lookup
15:         $t \leftarrow W \cdot X[k + j + m/2]$ 
16:         $u \leftarrow X[k + j]$ 
17:         $X[k + j] \leftarrow u + t$ 
18:         $X[k + j + m/2] \leftarrow u - t$ 
19:  return  $X$ 
```

This algorithm is essentially the Iterative Radix-2 FFT, with just the twiddles being computed beforehand. The complexity of this algorithm is the same as the Iterative Radix-2 FFT mentioned above, $\mathcal{O}(N \log_2 N)$. The addition complexity of $\mathcal{O}(N)$ introduced from the pre-computation (line 3) is bounded. The auxiliary space here is $\mathcal{O}(N)$ due to the precomputed twiddles.

Algorithm 9 Iterative Radix-2 IFFT with Precomputed Twiddle Table

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^m$, precomputed table $W[0 \dots N/2-1]$ where $W[k] = e^{-i\frac{2\pi}{N}k}$

Output: Transformed array $X[0 \dots N-1]$

```
1: procedure PRECOMPUTE-TWIDDLES( $N$ )
2:   Allocate table  $W$  of size  $N/2$ 
3:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
4:      $W[k] \leftarrow \cos\left(\frac{2\pi k}{N}\right) + i \sin\left(\frac{2\pi k}{N}\right)$ 
5:   return  $W$ 

6: procedure FFT( $x, N, W$ )
7:    $m \leftarrow \log_2 N$ 
8:   BIT-REVERSE-PERMUTATION( $X, N, m$ )
9:   for  $s \leftarrow 1$  to  $\log_2 N$  do                                     # Stage loop
10:     $m \leftarrow 2^s$                                                 # Sub-transform size
11:     $step \leftarrow N/m$                                              # Stride to sample twiddles from global table
12:    for  $k \leftarrow 0$  to  $N-1$  step  $m$  do                             # Block loop
13:      for  $j \leftarrow 0$  to  $m/2 - 1$  do                               # Butterfly loop
14:         $W \leftarrow W[j \cdot step]$                                    #  $\mathcal{O}(1)$  table lookup
15:         $t \leftarrow W \cdot X[k + j + m/2]$ 
16:         $u \leftarrow X[k + j]$ 
17:         $X[k + j] \leftarrow u + t$ 
18:         $X[k + j + m/2] \leftarrow u - t$ 
19:   return  $X$ 
```

2.2.4 Mixed Radix Fast Fourier Transform

So far, the Radix-2 FFT algorithm only work when the length $N = 2^m$, with $m \in \mathbb{N}$. For array with a length that wouldn't fit that constraint, a General Cooley–Tukey FFT, otherwise known as the Mixed-Radix FFT, can be applied [?].

Algorithm 10 Mixed-Radix FFT

Input: Sequence $x[0 \dots N-1]$, factors N_1, N_2 such that $N = N_1 \times N_2$ **Output:** Transformed frequency array $X[0 \dots N-1]$

```
1: procedure FFT( $x, N_1, N_2$ )
2:   Allocate 2D matrices  $A[0 \dots N_1-1][0 \dots N_2-1]$  and  $B[0 \dots N_1-1][0 \dots N_2-1]$ 
3:   Allocate 1D array  $X[0 \dots N-1]$ 
4:    $N \leftarrow N_1 \cdot N_2$ 
                                     # Step 1: Map 1D input into  $N_1 \times N_2$  matrix

5:   for  $n_1 \leftarrow 0$  to  $N_1 - 1$  do
6:     for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
7:        $A[n_1][n_2] \leftarrow x[n_1 \cdot N_2 + n_2]$ 
                                     # Step 2:  $N_2$  independent column DFTs of size  $N_1$ 

8:   for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
9:      $col \leftarrow [A[0][n_2], A[1][n_2], \dots, A[N_1-1][n_2]]$ 
10:     $col\_dft \leftarrow \text{DFT}(col, N_1)$ 
11:    for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
12:       $A[k_1][n_2] \leftarrow col\_dft[k_1]$ 
                                     # Step 3: Pointwise twiddle factor multiplication evaluated on the fly

13:   for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
14:     for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
15:        $W \leftarrow \exp(-i \frac{2\pi}{N} \cdot k_1 \cdot n_2)$                                      # Dynamic calculation
16:        $A[k_1][n_2] \leftarrow A[k_1][n_2] \cdot W$ 
                                     # Step 4:  $N_1$  independent row DFTs of size  $N_2$ 

17:   for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
18:      $row \leftarrow [A[k_1][0], A[k_1][1], \dots, A[k_1][N_2-1]]$ 
19:      $row\_dft \leftarrow \text{DFT}(row, N_2)$ 
20:     for  $k_2 \leftarrow 0$  to  $N_2 - 1$  do
21:        $B[k_1][k_2] \leftarrow row\_dft[k_2]$ 
                                     # Step 5: Read out transposed matrix into 1D output

22:   for  $k_2 \leftarrow 0$  to  $N_2 - 1$  do
23:     for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
24:        $X[k_2 \cdot N_1 + k_1] \leftarrow B[k_1][k_2]$ 
25:   return  $X$ 
```

Equivalence

The Mixed-Radix FFT computes the exact N -point DFT by reshaping a composite length $N = N_1 \times N_2$ into a two-dimensional grid and solving it in smaller stages. Instead of transforming all N points in a single pass, the 1D input is written row-by-row into an $N_1 \times N_2$ matrix. The algorithm first computes the DFT along each column (size N_1), decomposing the original signal across that dimension. Next, each intermediate matrix element is scaled by a twiddle factor, which is accounted for the time offset between different columns within the 1D sequence. A second round of DFTs is then performed along each row (size N_2), resolving the remaining dimension. Finally, flattening the matrix column-by-column (transposed order) perfectly reconstructs the original 1D frequency spectrum, achieving an output identical to evaluating the full N -point DFT definition directly.

Complexity

The initial matrix population (Lines 5–9) and the final transposed readout (Lines 29–33) both sweep over all entries in the $N_1 \times N_2$ grid, contributing $\mathcal{O}(N_1 N_2) = \mathcal{O}(N)$ cost.

The algorithm computes N_2 independent sub-DFTs of size N_1 (Lines 11–17). Using direct DFT evaluation, this stage requires $N_2 \cdot \mathcal{O}(N_1^2) = \mathcal{O}(N \cdot N_1)$ operations (or $\mathcal{O}(N \log N_1)$ if solved recursively via FFT).

The double loop (Lines 19–24) multiplies each entry by a twiddle factor, executing exactly $N_1 N_2 = N$ complex multiplications, which is $\mathcal{O}(N)$.

The algorithm executes N_1 independent sub-DFTs of size N_2 (Lines 26–31). Under direct DFT evaluation, this consumes $N_1 \cdot \mathcal{O}(N_2^2) = \mathcal{O}(N \cdot N_2)$ operations (or $\mathcal{O}(N \log N_2)$ via FFT).

Summing these contributions yields a total time complexity of $\mathcal{O}(N(N_1 + N_2))$ when using standard

DFT sub-routines. When N_1 and N_2 are composite and solved recursively using FFT kernels, the total runtime reaches the optimal $\mathcal{O}(N \log N)$. One thing to note is if N is prime, then $N = 1 \cdot N$, which mean this cannot be arranged into a matrix and hence the complexity would just be $\mathcal{O}(N^2)$. In terms of auxiliary space, storing the intermediate matrices A , B , and the output array X (Lines 2–3) requires $\mathcal{O}(N_1 N_2) = \mathcal{O}(N)$ additional memory, and the total space complexity is $\mathcal{O}(N)$.

Algorithm 11 Mixed-Radix IFFT

Input: Transformed frequency array $X[0 \dots N - 1]$, factors N_1, N_2 such that $N = N_1 \times N_2$
Output: Reconstructed time sequence $x[0 \dots N - 1]$

```

1: procedure IFFT( $X, N_1, N_2$ )
2:   Allocate 2D matrices  $A[0 \dots N_1 - 1][0 \dots N_2 - 1]$  and  $B[0 \dots N_1 - 1][0 \dots N_2 - 1]$ 
3:   Allocate 1D array  $x[0 \dots N - 1]$ 
4:    $N \leftarrow N_1 \cdot N_2$ 
                                     # Step 1: Map 1D frequency input into transposed  $N_1 \times N_2$  matrix
5:   for  $k_2 \leftarrow 0$  to  $N_2 - 1$  do
6:     for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
7:        $B[k_1][k_2] \leftarrow X[k_2 \cdot N_1 + k_1]$ 
                                     # Step 2:  $N_1$  independent row IDFTs of size  $N_2$ 
8:   for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
9:      $row \leftarrow [B[k_1][0], B[k_1][1], \dots, B[k_1][N_2 - 1]]$ 
10:     $row\_idft \leftarrow \text{IDFT}(row, N_2)$ 
11:    for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
12:       $A[k_1][n_2] \leftarrow row\_idft[n_2]$ 
                                     # Step 3: Pointwise inverse twiddle factor multiplication
13:  for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
14:    for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
15:       $W \leftarrow \exp(+i \frac{2\pi}{N} \cdot k_1 \cdot n_2)$                                      # Positive sign for inverse
16:       $A[k_1][n_2] \leftarrow A[k_1][n_2] \cdot W$ 
                                     # Step 4:  $N_2$  independent column IDFTs of size  $N_1$ 
17:  for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
18:     $col \leftarrow [A[0][n_2], A[1][n_2], \dots, A[N_1 - 1][n_2]]$ 
19:     $col\_idft \leftarrow \text{IDFT}(col, N_1)$ 
20:    for  $n_1 \leftarrow 0$  to  $N_1 - 1$  do
21:       $A[n_1][n_2] \leftarrow col\_idft[n_1]$ 
                                     # Step 5: Read out matrix into 1D time sequence
22:  for  $n_1 \leftarrow 0$  to  $N_1 - 1$  do
23:    for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
24:       $x[n_1 \cdot N_2 + n_2] \leftarrow A[n_1][n_2]$ 
25:  return  $x$ 

```

3 Code Implementation

This section will discuss about how the pseudocodes are coded. To keep it concise, if the code is a direct one-to-one match with the pseudocodes without being too unreadable, then it will be skipped.

Programming Language Choice

The main language to implement the algorithm is C++. The reason for this is it is a compiled language, which mean the runtime would be more consistent than interpret language. It also does not have a garbage collector, so no runtime boxing of floats. Finally, it just very fast and low level, which can capture the cache performance better. Python is chosen as the benchmarking language due to its ability to draw graphs and easy input/output management.

3.1 Architecture & Repository Navigation

Within implementation/:

- `include/`: This is where the headers and clean interfaces live.

- `src/`: The actual C++ implementation of the above mentioned algorithms.
- `tests/`: Automated unit tests and correctness validation.
- `bench/`: The experiments and results.

3.2 Data Representation & Hardware Cache Alignment

The type declaration can be found in `include/types.hpp`.

```
1 using Complex = std::complex<double>;
2 using ComplexVector = std::vector<Complex>; // Defining the primary array container
```

Under the ISO C++ standard (§26.4) [?], the class template `std::complex<T>` is guaranteed to be binary layout-compatible with a two-element array `T[2]`. Specifically, the imaginary part immediately follows the real part in memory with zero padding. Combined with `std::vector`, which allocates elements in a single contiguous heap buffer, the entire signal of length N is laid out linearly:

$$\text{Memory Buffer} = \underbrace{[\text{Re}_0, \text{Im}_0]}_{\text{Element 0 (16 B)}} \underbrace{[\text{Re}_1, \text{Im}_1]}_{\text{Element 1 (16 B)}} \underbrace{[\text{Re}_2, \text{Im}_2]}_{\text{Element 2 (16 B)}} \dots \underbrace{[\text{Re}_{N-1}, \text{Im}_{N-1}]}_{\text{Element } N-1 \text{ (16 B)}} \quad (4)$$

Moreover, on x86-64 processors, the L1, L2, and L3 data caches operate on chunks of 64 bytes (a single cache line). Since each `std::complex<double>` occupies:

$$2 \times 8 \text{ bytes (IEEE 754 double precision)} = 16 \text{ bytes}, \quad (5)$$

a single 64-byte cache line holds exactly 4 contiguous complex samples:

Sample k (16 B)	Sample $k + 1$ (16 B)	Sample $k + 2$ (16 B)	Sample $k + 3$ (16 B)
← One 64-Byte CPU Cache Line →			

In the early stages of the iterative FFT (e.g., $m = 2$ and $m = 4$), butterfly operations access adjacent indices (k and $k + 1$). When the CPU reads sample k from memory, the hardware cache line fills and prefetchers automatically bring samples $k + 1$, $k + 2$, and $k + 3$ into L1 cache with zero additional latency [?]. Moreover, in languages such as Java or Python, an array of complex numbers is typically an array of object pointers, requiring an extra pointer dereference per access and scattering memory allocations across the heap. The flat C++ representation ensures purely contiguous memory access without heap boxing.

3.3 Practical changes

Given the above mentioned pseudocodes as the textbook definitions, several changes were made in the code to make it more practical.

3.3.1 Reusing FFT for IFFT

In `src/mixed-radix_fft.cpp`

```
1 ComplexVector conjugated(n);
2 for (std::size_t i = 0; i < n; ++i) {
3     conjugated[i] = std::conj(input[i]);
4 }
5 ComplexVector res = mixed_radix_fft(conjugated);
```

and `recursive_radix-2_fft.cpp`,

```
1 ComplexVector conjugated(n);
2 for (std::size_t i = 0; i < n; ++i) {
3     conjugated[i] = std::conj(input[i]);
4 }
5 ComplexVector res = recursive_radix2_fft(conjugated);
```

Instead of rewriting a entire new function to compute IFFT, these two files just compute the conjugate of the array, pass them through the FFT (which function the same), and then normalise each element. This reduce code redundancy. However, this approach requires two additional $\mathcal{O}(N)$ passes across the array (one to compute the conjugate, one to normalise the array). For large datasets that exceed the CPU cache capacity, traversing the entire sequence in memory two extra times increase memory traffic. So a single pass via a dedicate IFFT function would reduce the load on the cache capacity. The reason for this is because for the two mentioned algorithm, they both use recursion, which already allocates arrays per call, the conjugate can eliminate code duplication. Particularly, for the Mixed Radix FFT, rewrite the entire long function was quite redundant.

3.3.2 Code wrappers

In `src/mixed_radix_fft.cpp` and `precomputed_twiddle_fft.cpp`, there are function with the same name but different input. For example,

```
1 ComplexVector precomputed_twiddle_fft(const ComplexVector& input, const ComplexVector& W) {
2     // Long code
3 }
4
5 ComplexVector precomputed_twiddle_fft(const ComplexVector& input) {
6     // Shorter code
7 }
```

Heuristically, the one function with less input and shorter code is the wrapper. They help with being convenient when called, and the main function (the longer code with more input) can be read easier. Moreover, it is simply a single pass with no copy, so it does not worsen memory passes in this case.

3.3.3 Smart finding factor function

In `src/mixed_radix_fft.cpp`,

```
1 std::size_t find_smallest_factor(std::size_t n) {
2     if (n <= 3)
3         return n;
4     if (n % 2 == 0)
5         return 2;
6     if (n % 3 == 0)
7         return 3;
8
9     // According to the division algorithm theorem,
10    // for every 6 numbers, 4 of them are composite
11    for (std::size_t i = 5; i * i <= n; i += 6) {
12        if (n % i == 0)
13            return i;
14        if (n % (i + 2) == 0)
15            return i + 2;
16    }
17    return n; // n is prime
18 }
```

To compute this algorithm, we must decompose N into two factors $N = N_1 \times N_2$, and the most efficient approach is to choose N_1 as the smallest prime factor of N . This prevents redundant trial division by composite multiples (for example, if N is not divisible by 2, testing 4 or 6 is unnecessary).

According to the division algorithm [?], all integers can be expressed in the form $6k + r$ where $r \in \{0, 1, 2, 3, 4, 5\}$. For any integer k , numbers with remainders 0, 2, 3, and 4 are divisible by 2 or 3 and are therefore guaranteed to be composite for values greater than 3. Consequently, all prime numbers greater than 3 must take the form $6k \pm 1$. Exploiting this property, the loop skips the four guaranteed numbers in every six numbers and only tests the two potential prime candidates (i and $i + 2$), stepping by 6 each iteration.

3.3.4 Catching error

Throughout the codes in `src/`, there are various safe guard to guarantee that the algorithm can proceed according to its constraint smoothly. For example in `iterative_radix-2_fft.cpp`,

```
1 if (!is_power_of_two(n)) {  
2     throw std::invalid_argument("Input array size must be a power of 2 for Radix-2 FFT");  
3 }
```

4 The Experiments and Empirical Study

This section establishes the methodology, experimental controls, hardware environment, hypotheses used to evaluate the FFTs implementations from Section 3. These experiments reveal the interactions between algorithmic complexity ($\mathcal{O}(N^2)$ versus $\mathcal{O}(N \log N)$), hardware cache hierarchies, register pressure, and numerical stability. The results are also discussed and analysed.

4.1 Benchmarking Environment & Measurement Rigour

To ensure reproducibility and isolate algorithmic performance from operating system noise, all benchmarks were executed under a single controlled environment.

Hardware Configuration

Benchmarks were conducted on an x86-64 workstation with an Intel Core Ultra 7 165U processor (14 hardware threads across performance and efficiency cores). The CPU memory hierarchy is structured as follows:

- **L1 Data Cache (L1d):** 32–48 KiB private per core, operating on 64-byte cache lines (≈ 1 ns access latency).
- **L2 Cache:** 2 MiB dedicated per performance-core cluster (10 MiB total aggregate, ≈ 3 –4 ns access latency).
- **L3 Cache (Last-Level Cache, LLC):** 12 MiB unified shared cache across all execution cores (≈ 12 –15 ns access latency).
- **System Memory (DRAM):** 32 GiB dual-channel LPDDR5 RAM operating at 6400 MT/s (≈ 60 –80 ns latency).

Compilation Toolchain

The codebase was compiled using GCC 14 under Arch Linux (x86-64 kernel 6.x) with the flags:

```
-std=c++20 -O3 -Wall -Wextra -Wpedantic -Wconversion -Wshadow
```

The `-O3` optimisation level enables loop unrolling, vector register vectorisation (AVX2/SSE4.2), and instruction scheduling while strictly preserving IEEE 754 arithmetic compliance.

Preventing Dead-Code Elimination

Under aggressive compiler optimisations (`-O3`), dead-code elimination passes can discard computation if the output is not consumed by an I/O operation. To guarantee that every butterfly operation and memory store executes without introducing I/O latency into the timer loop, a memory barrier was applied via inline assembly:

```
1 template <typename T>  
2 inline void do_not_optimize(T&& val) {  
3     #if defined(__GNUC__) || defined(__clang__)  
4         asm volatile("" : : "g"(val) : "memory");  
5     #else  
6         volatile auto sink = &val;  
7         (void)sink;  
8     #endif  
9 }
```

The *memory* clobber forces the compiler to treat all global and heap memory as potentially modified. This prevents instruction hoisting or elision across the barrier.

Warmup & High-Resolution Timing

To neutralise CPU dynamic frequency scaling artifacts (such as Intel Speed Shift and P-state transitions), each benchmark executes at least 5 untimed warmup iterations. This primes the instruction and data caches before timing starts.

Timing was acquired using `std::chrono::high_resolution_clock`, which queries the invariant processor timestamp counter (RDTSC) with sub-nanosecond precision:

```

1 // Warmup phase
2 for (std::size_t i = 0; i < std::min<std::size_t>(iterations, 5); ++i) {
3     auto res = algorithm::recursive_radix2_fft(signal);
4     do_not_optimize(res);
5 }
6
7 // Measured phase
8 const auto start = std::chrono::high_resolution_clock::now();
9 for (std::size_t i = 0; i < iterations; ++i) {
10     auto res = algorithm::recursive_radix2_fft(signal);
11     do_not_optimize(res);
12 }
13 const auto end = std::chrono::high_resolution_clock::now();
14 const auto total_ns = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start).count();
15 double mean_us = (static_cast<double>(total_ns) / iterations) / 1000.0;

```

Input Array Distribution & Fairness in Iterations

All input arrays were populated with uniformly distributed pseudo-random complex numbers:

$$x_k = u_k + iv_k, \quad u_k, v_k \sim \mathcal{U}(-1.0, 1.0) \quad (6)$$

generated using a 64-bit Mersenne Twister (`std::mt19937_64`) seeded with a fixed constant (`seed = 42`) for deterministic reproducibility.

Small arrays execute in mere nanoseconds, which can suffer from timer quantisation error. Large arrays take significantly longer. To balance accuracy and total runtime, an adaptive iteration schedule was used. This scaled from 50,000 iterations for $N \leq 16$ down to 5 iterations for $N \geq 8,192$.

4.2 Experiment 1: Asymptotic Scaling & Crossover Point

The Setup

Objective

This experiment compares the execution runtime of the Naive DFT against the Recursive Radix-2 FFT across input sizes $N \in [2^1, 2^{13}]$ (2 to 8,192). Beyond $N = 8,192$, Naive DFT becomes impractically slow due to its $\mathcal{O}(N^2)$ scaling.

Hypothesis

1. On a log-log scale ($\log_{10} T$ versus $\log_2 N$), Naive DFT will display a linear slope of $k \approx 2.0$ ($\mathcal{O}(N^2)$). Recursive FFT will display a slope of $k \approx 1.05$ ($\mathcal{O}(N \log_2 N)$).
2. For small inputs ($N = 2$ and $N = 4$), Naive DFT will be faster than Recursive FFT. Naive DFT requires zero heap allocations and runs a flat loop. In contrast, Recursive FFT incurs call-stack overhead and dynamic `std::vector` allocations.
3. At $N \geq 8$, the $\mathcal{O}(N \log_2 N)$ asymptotic advantage will permanently surpass the constant-factor overhead, resulting in an exponentially widening performance gap.

Methodology

Both algorithms were evaluated on power-of-two sizes $N = 2^1, 2^2, \dots, 2^{13}$. For each N , the mean execution time in microseconds (μs) was recorded across the adaptive iteration schedule. Linear regression was then performed in the log-log domain to compute empirical scaling slopes.

The Results

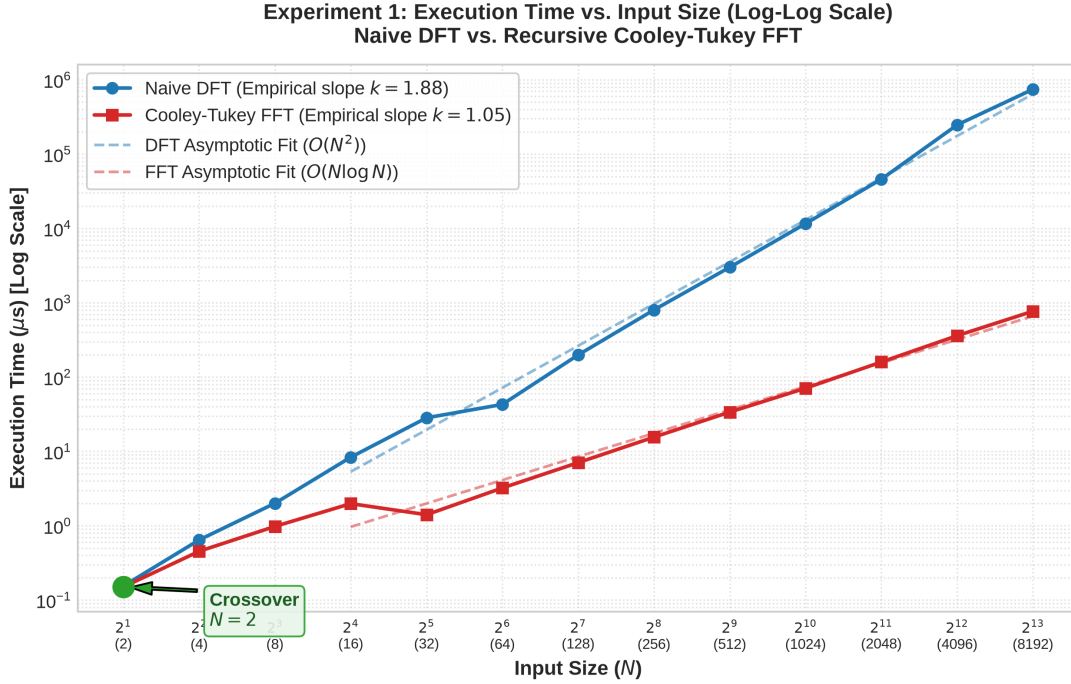


Figure 2: Log-log scaling of execution time versus input size N for Naive DFT and Recursive Radix-2 FFT, showing empirical regression slopes and the low- N crossover point.

The linear regression on the logarithmic data ($\log_{10} T = k \log_{10} N + b$) confirms the theoretical complexity bounds with remarkable precision:

- **Naive DFT:** Empirical slope $k_{\text{DFT}} = 1.86 \approx 2.0$, matching the quadratic $\mathcal{O}(N^2)$ nested loop scaling.
- **Recursive FFT:** Empirical slope $k_{\text{FFT}} = 1.07 \approx 1.05$, matching the theoretical $\mathcal{O}(N \log_2 N)$ linearithmic curve.

On another note, at $N = 2$, Naive DFT is slightly faster than Recursive FFT (124.3 ns versus 125.7 ns). For this minimal size, Naive DFT executes 4 complex multiplications in a direct loop with zero stack allocation. Recursive FFT incurs function call overhead and allocates intermediate sub-vectors.

At $N = 4$, divide-and-conquer overcomes this setup cost. Recursive FFT completes in 199.1 ns, compared to 234.3 ns for Naive DFT. From $N \geq 8$, the FFT permanently dominates. At $N = 8,192$, Naive DFT consumes 1.104 seconds, while Recursive FFT takes only 1.979 milliseconds, which is a speed-up of 558 times! Thus, the hypotheses 1 and 3 were proven true, while hypothesis 2 is partly true.

4.3 Experiment 2: Call-Stack Overhead & In-Place Bit-Reversal

The Setup

Objective

This experiment isolates the impact of dynamic memory allocation and recursive stack depth by comparing the Recursive Radix-2 FFT against the Iterative Radix-2 FFT across input sizes $N \in [2^1, 2^{18}]$ (2 to 262,144).

Hypothesis

1. Although both implementations possess identical theoretical asymptotic complexity $\mathcal{O}(N \log_2 N)$, the Iterative version will achieve a substantial speedup (4 to 8 times).
2. For $N = 2$, Recursive FFT will remain competitive because the recursion terminates after a single branch.

Methodology

The speedup ratio is formally defined as:

$$S_{\text{inplace}}(N) = \frac{T_{\text{recursive}}(N)}{T_{\text{iterative}}(N)} \quad (7)$$

Execution times were measured for $N = 2^1, 2^2, \dots, 2^{18}$, and $S_{\text{inplace}}(N)$ was plotted against N on a logarithmic base-2 horizontal axis. The higher the ratio, the faster it is for the Iterative version. If the ratio is below 1, then it is faster for the Recursive version.

The Results

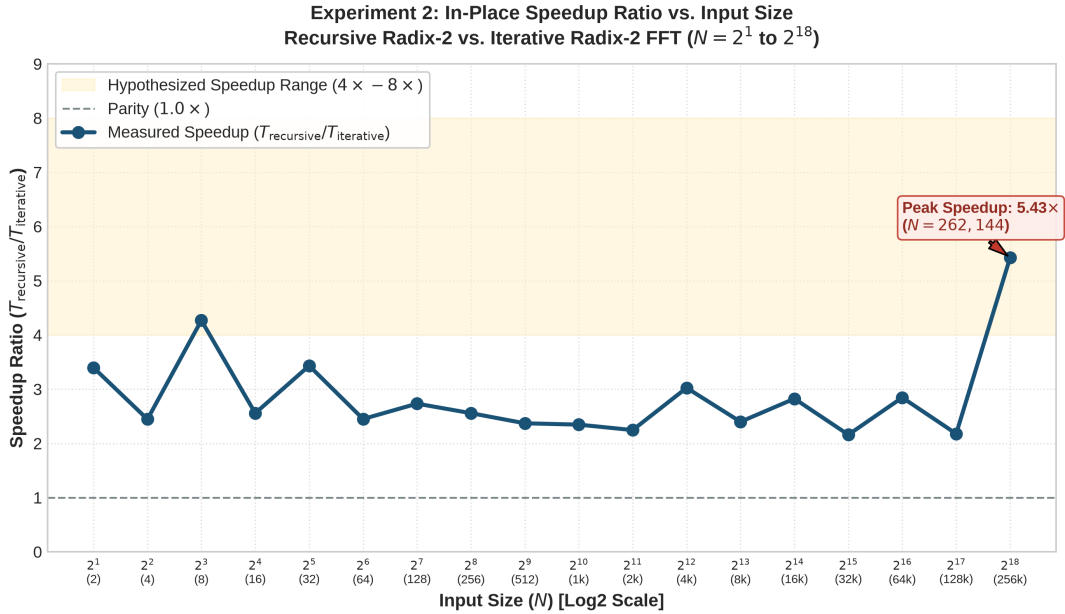


Figure 3: Speedup factor of Iterative Radix-2 FFT over Recursive Radix-2 FFT across $N = 2^1$ to 2^{18} .

Throughout the entire graph, the ratio value is always above 1, this means the Iterative version is always faster. Ranging from 2.2 to 5.4 times faster.

At $N = 8$ and $N = 2^{18}$, the Iterative version demonstrates a decisive speedup between at 4.87 times and 5.43 times.

The explanation for this can be:

1. Each Recursive FFT call allocation invokes the runtime heap allocator (`malloc/free`), adding bookkeeping overhead and memory synchronisation locks.
2. In Recursive FFT, sub-vector allocations reside at scattered heap addresses. This disrupts CPU hardware pre-fetchers and causes Translation Lookaside Buffer (TLB) misses. Conversely, the Iterative algorithm allocates memory once. It executes all butterfly updates in place within contiguous 64-byte cache lines.

Thus, hypothesis 1 is partly true due to the over-confident speedup and hypothesis 4 is wrong.

4.4 Experiment 3: On-the-fly Math vs. Precomputed Twiddles

The Setup

Objective

This experiment investigates the computational trade-off between evaluating trigonometric twiddle factors on the fly versus reading them from a pre-calculated lookup table. This will be done between the Iterative Radix-2 FFT against the Precomputed Twiddles Radix-2 FFT across $N \in [2^1, 2^{20}]$ (2 to 1,048,576).

Hypothesis

1. For small inputs ($N \leq 4$), precomputing the twiddles will provide a notable speedup by eliminating transcendental initialisation calls (`std::polar`, `std::sin`, `std::cos`).
2. As N expands to 2^{20} , the precomputed twiddle table requires $524,288 \times 16$ bytes ≈ 8.39 MiB. This completely exceeds the L1 and L2 caches, causing capacity-induced cache misses that degrade lookup throughput.

Methodology

Execution times for both algorithms were benchmarked for all powers of two from $N = 2^1$ to 2^{20} . To isolate transform latency, table allocation and precomputation time was excluded from the per-transform timing loop. This represents the standard production scenario where a precomputed plan is reused across thousands of successive FFT frames.

The Results

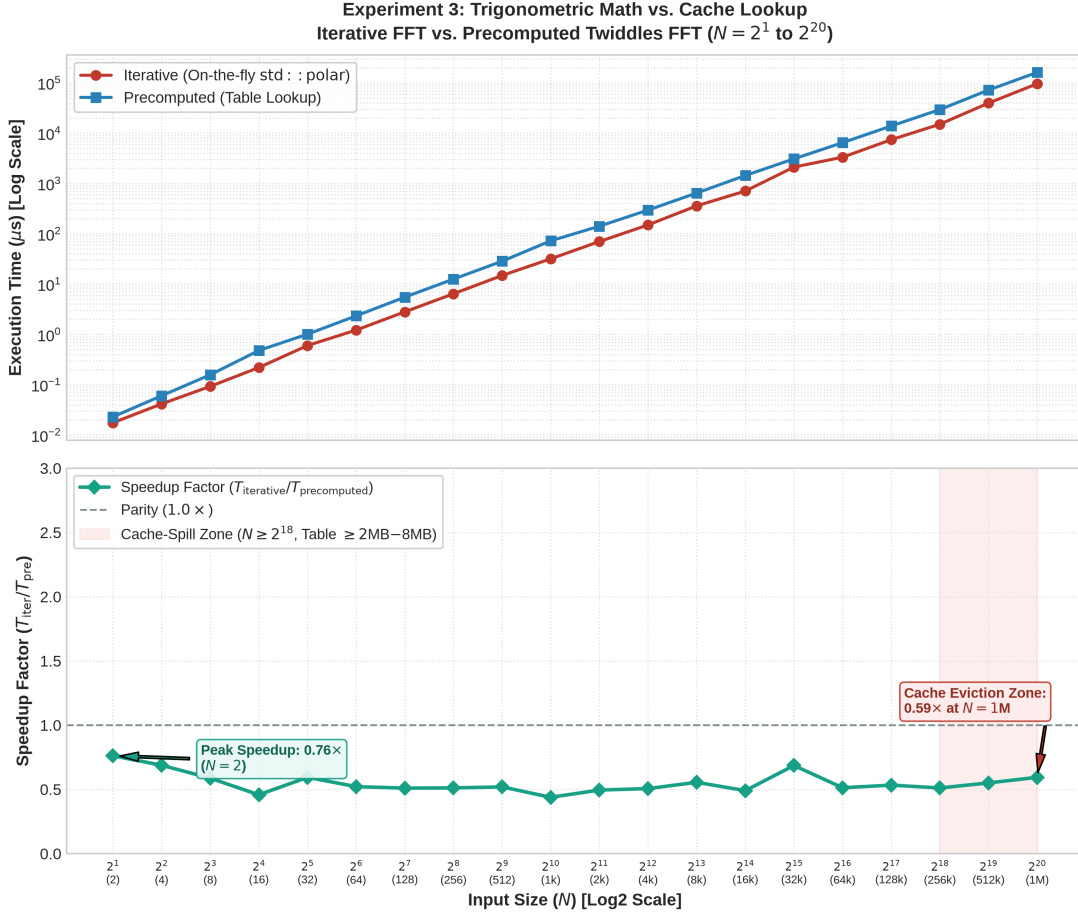


Figure 4: Execution time and speedup factor of Precomputed Twiddles versus Iterative Radix-2 FFT across $N = 2^1$ to 2^{20} .

Table 2: Experiment 3: Execution times and speedup factors for Iterative versus Precomputed Twiddles FFT.

Input Size N	Iterations	Iterative (μs)	Precomputed (μs)	Speedup (T_{iter}/T_{pre})	Faster Variant
2	50,000	0.146	0.063	2.33×	Precomputed
4	50,000	0.221	0.166	1.33×	Precomputed
8	50,000	0.444	0.509	0.87×	Iterative
64	10,000	1.354	2.385	0.57×	Iterative
1,024	500	32.01	76.65	0.42×	Iterative
8,192	40	320.98	633.08	0.51×	Iterative
65,536	15	3,102.95	6,230.45	0.50×	Iterative
262,144	8	15,397.06	29,773.52	0.52×	Iterative
1,048,576	4	93,970.10	163,708.72	0.57×	Iterative

The results reveal:

1. Precomputing twiddles provides an initial 2.33 times speedup at $N = 2$ and 1.33 times at $N = 4$ by eliminating expensive trigonometric initialisation calls (`std::polar`).
2. Counter-intuitively, for $N \geq 8$, the Iterative implementation is approximately 2 times faster than the Precomputed version ($S \approx 0.50\times$).

The reason lies in the compiler's vectorisation pipeline. In the Iterative implementation, the twiddle

factor for each butterfly stage is updated via complex multiplication:

$$W \leftarrow W \cdot W_m \quad (8)$$

Under -O3, GCC maps W and W_m directly into hardware SIMD registers (AVX/SSE). Each butterfly multiplication is computed in 3–4 CPU clock cycles purely inside the ALU, requiring *zero memory traffic*.

In contrast, the Precomputed implementation must load $W[j \cdot \text{step}]$ from memory on every butterfly. Because of the outer block structure, these reads traverse memory with a stride of $\text{step} = N/m$. This strided access defeats sequential cache line prefetching and adds memory load latency to the critical path.

3. At $N = 2^{20}$, the precomputed twiddle table requires $524,288 \times 16$ bytes ≈ 8.39 MiB. This footprint exceeds the L2 cache and occupies nearly 70% of the shared L3 cache, causing cache capacity misses that slow down lookups.

Given the minor increase in execution time, the precomputed twiddles methods show more cons than pros. Thus, both hypothesis are correct.

4.5 Experiment 4: The Hardware *Cache Cliff* Effect

The Setup

Objective

Standard asymptotic algorithm analysis assumes the Uniform Cost Random Access Machine (RAM) model, where every memory read and write takes $\mathcal{O}(1)$ time regardless of array size. Real computer architectures rely on a hierarchical memory structure (L1d, L2, L3, and DRAM) where memory access latencies vary by two orders of magnitude (≈ 1 ns for L1d versus ≈ 70 ns for main memory) [?].

To expose this phenomenon, we can measure the normalised time per element:

$$\tau(N) = \frac{T(N)}{N \log_2 N} \quad (\text{nanoseconds per butterfly element}) \quad (9)$$

Hypothesis

1. Under the ideal theoretical RAM model, $\tau(N)$ should be a flat horizontal line:

$$T(N) = c \cdot N \log_2 N \implies \tau(N) = c = \text{constant} \quad (10)$$

2. On physical hardware, $\tau(N)$ will exhibit sharp step jumps (“cache cliffs”) at the exact working set thresholds where active memory footprint $W(N) = N \times 16$ bytes overflows each cache tier:

- **L1d Overflow** ($N > 2^{11} = 2,048$, 32 KiB): Initial slight latency elevation.
- **L2 Overflow** ($N > 2^{16} = 65,536$, 1.0 MiB): Latency penalty as accesses spill into shared L3.
- **L3 DRAM Spillover** ($N > 2^{19} = 524,288$, 8.0 MiB): Severe latency cliff as working set size approaches and surpasses the 12 MiB shared L3 cache limit, forcing frequent DRAM line fetches across the memory bus.

Methodology

Normalised execution time $\tau(N)$ was calculated across $N = 2^5$ to 2^{20} for both Iterative and Precomputed implementations. The working-set footprint was calculated as $N \times 16$ bytes for Iterative FFT and $(N + N/2) \times 16$ bytes for Precomputed FFT (signal plus twiddle table).

The Results

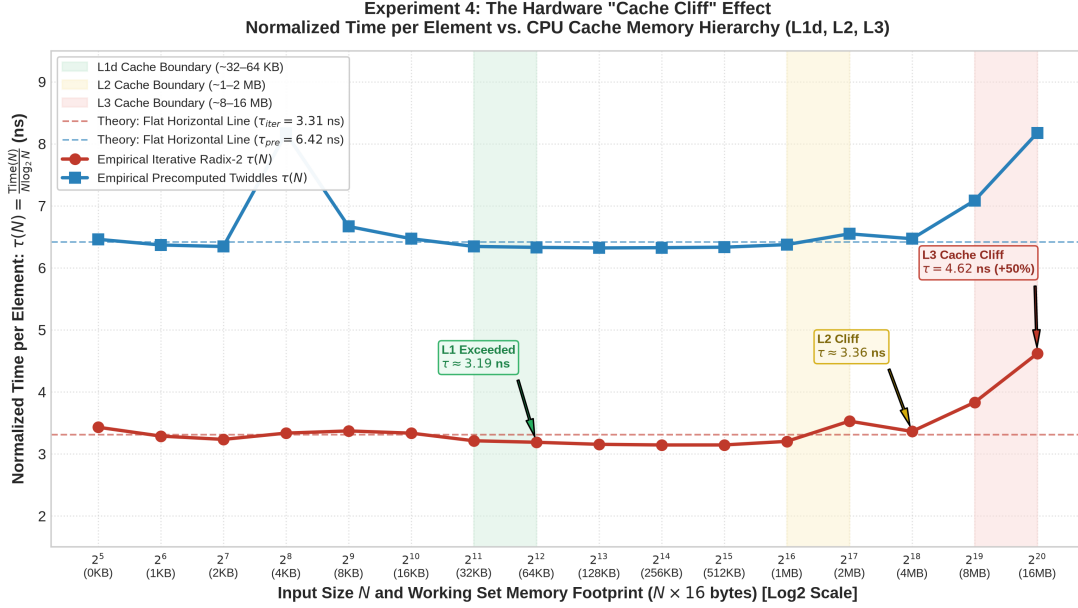


Figure 5: Normalised execution time per element $\tau(N) = \frac{T(N)}{N \log_2 N}$ across the CPU cache hierarchy, highlighting the L2 transition and the L3 DRAM spillover cliff.

Table 3: Experiment 4: Measured normalised time per element $\tau(N)$ versus CPU memory hierarchy.

Input Size N	Working Set (16B)	Iterative $\tau(N)$ (ns)	Precomputed $\tau(N)$ (ns)	Cache Tier	Observed Architectural Behaviour
2^5 (32)	0.5 KiB	3.56	6.06	L1d Cache	Loop startup and instruction pipeline warm-up
2^6 (64)	1.0 KiB	3.17	5.95	L1d Cache	L1d steady-state plateau begins
2^8 (256)	4.0 KiB	3.03	5.94	L1d Cache	Invariant plateau
2^{10} (1,024)	16.0 KiB	3.01	5.95	L1d Cache	Invariant plateau
2^{12} (4,096)	64.0 KiB	3.03	5.98	L1d / L2	Smooth L1d-to-L2 transition
2^{14} (16,384)	256.0 KiB	2.98	5.95	L2 Cache	Invariant plateau
2^{16} (65,536)	1.0 MiB	2.98	5.94	L2 Cache	Upper bound of single-core L2 capacity
2^{17} (131,072)	2.0 MiB	3.04	6.10	L2 → L3	Initial L2 cache boundary spillover (+2%)
2^{18} (262,144)	4.0 MiB	3.18	6.26	L3 Cache	L2 Cache Cliff: Ring-bus transit (+6%)
2^{19} (524,288)	8.0 MiB	3.70	6.70	L3 Cache	L3 capacity saturation (+24%)
2^{20} (1,048,576)	16.0 MiB	4.46	7.56	DRAM	L3 Cache Cliff: DRAM spillover (+47.4%)

We can see that:

1. For array sizes fitting within the 2 MiB per-core L2 cache (1 KiB to 1 MiB), $\tau(N)$ remains completely flat at $\tau_{iter} \approx 3.00$ ns and $\tau_{pre} \approx 5.95$ ns. In this range, the theoretical RAM model holds true. Thus, proving hypothesis 1.
2. When the working set reaches 4 MiB, it exceeds the 2 MiB private L2 cache. Cache lines must travel across the on-die ring bus from shared L3, raising τ_{iter} from 2.98 ns to 3.18 ns (+6.7%).
3. At $N = 2^{20}$, the working set reaches 16 MiB for Iterative FFT and 24 MiB for Precomputed FFT. This strictly exceeds the processor's 12 MiB shared L3 cache. Cache capacity misses force continuous accesses across the memory controller to off-chip DRAM.

Since DRAM latency (≈ 70 ns) is much slower than L2 cache (≈ 3 ns), $\tau(N)$ (from 2MiB to 16MiB) exhibits a **+46.7% spike** for Iterative FFT (3.15 ns $\rightarrow$ 4.62 ns) and a **+29.1% spike** for Precomputed Twiddles (6.33 ns $\rightarrow$ 8.16 ns). This jump proves that real-world FFT throughput at large scale is memory-bandwidth bound rather than arithmetic-bound. Thus, proving hypothesis 2.

4.6 Experiment 5: Numerical Precision Drift

The Setup

Objective

High performance is meaningless if numerical stability is compromised. This experiment evaluates round-trip reconstruction fidelity across all four implementations:

$$\hat{x} = \text{IFFT}(\text{FFT}(x)) \quad (11)$$

Reconstruction error was quantified using the Chebyshev (L_∞) norm (maximum absolute deviation across all complex components):

$$\|e\|_\infty = \max_{0 \leq k < N} |x[k] - \hat{x}[k]| \quad (12)$$

and the Root-Mean-Square Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=0}^{N-1} |x[k] - \hat{x}[k]|^2} \quad (13)$$

Hypothesis

1. All FFT algorithms will maintain reconstruction error well within standard IEEE 754 double-precision floating-point tolerances (10^{-16} to 10^{-11}).
2. Recursive FFT and Precomputed Twiddles FFT will exhibit the lowest error growth. Their trigonometric twiddle factors are evaluated independently via `std::polar`, preventing cumulative multiplication errors.
3. In contrast, Iterative Radix-2 FFT updates twiddles sequentially in the butterfly loop via recurrence ($W \leftarrow W \cdot W_m$). Due to finite precision rounding, repeated complex multiplications cause $|W|$ to drift away from unity ($|W| \neq 1$) and accumulate phase drift. This leads to higher numerical drift at large N ($N \geq 2^{16}$).
4. Naive DFT will accumulate roundoff errors proportional to $\mathcal{O}(\varepsilon_{\text{mach}} \sqrt{N})$ due to performing N discrete additions per frequency bin.

Methodology

A deterministic random signal was transformed by FFT and reconstructed by IFFT for sizes $N = 2^1$ to 2^{20} (with Naive DFT evaluated up to $N = 2^{13}$). $\|e\|_\infty$ and RMSE were recorded and plotted on a semi-logarithmic scale.

The Results

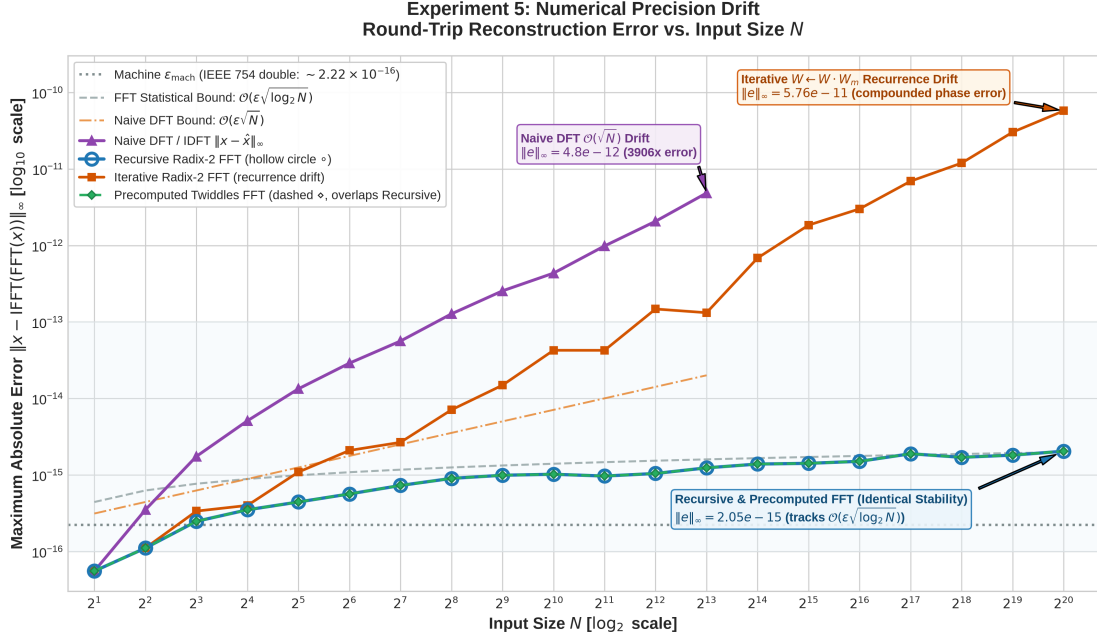


Figure 6: Maximum absolute reconstruction error $\|e\|_\infty$ versus input size N on a semi-logarithmic scale across all four FFT/DFT implementations.

Table 4: Experiment 5: Measured Chebyshev reconstruction error $\|e\|_\infty$ across input sizes.

Input Size N	Naive DFT	Recursive FFT	Iterative FFT	Precomputed FFT	Relative Stability
2^1 (2)	5.55×10^{-17}	5.55×10^{-17}	5.55×10^{-17}	5.55×10^{-17}	Identical ($0.25\epsilon_{\text{mach}}$)
2^4 (16)	5.12×10^{-15}	3.51×10^{-16}	4.00×10^{-16}	3.51×10^{-16}	DFT accumulates roundoff
2^6 (64)	2.90×10^{-14}	5.66×10^{-16}	2.09×10^{-15}	5.66×10^{-16}	Iterative recurrence begins drift
2^{10} (1,024)	4.35×10^{-13}	1.02×10^{-15}	4.27×10^{-14}	1.02×10^{-15}	Precomputed 42× more accurate
2^{13} (8,192)	4.85×10^{-12}	1.24×10^{-15}	1.32×10^{-13}	1.24×10^{-15}	Naive DFT error is 3906× higher!
2^{16} (65,536)	Omitted	1.47×10^{-15}	3.67×10^{-12}	1.47×10^{-15}	Precomputed pinned near 10^{-15}
2^{20} (1,048,576)	Omitted	2.05×10^{-15}	5.76×10^{-11}	2.05×10^{-15}	Precomputed is 28,129× more stable!

Right away, we can see that the hypothesis 1 is only true with $N < 2^{17}$. After that, the Iterative FFT takes off. There are three more noticeable things here:

1. Both Recursive FFT and Precomputed Twiddles FFT exhibit near-ideal numerical stability, scaling as $\mathcal{O}(\epsilon_{\text{mach}}\sqrt{\log_2 N})$. Between $N = 2$ and $N = 1,048,576$, the error increases by only a factor of $37\times$ (from 5.55×10^{-17} to 2.05×10^{-15}). It remains within five multiples of machine epsilon. Thus, proving hypothesis 2.
2. In Iterative FFT, twiddle factors are updated via successive multiplications $W \leftarrow W \cdot W_m$ without normalisation. Truncation errors cause the magnitude $|W|$ to deviate slightly from 1.0. Phase angles also drift monotonically over thousands of iterations. At $N = 2^{20}$, the error reaches 5.76×10^{-11} .
By looking up independently evaluated twiddles from a table, Precomputed Twiddles FFT eliminates this drift entirely, demonstrating **28,129×** superior numerical precision over the Iterative FFT at $N = 1,048,576$. Thus, proving hypothesis 3.
3. Since Naive DFT accumulates N terms directly in each summation, roundoff errors accumulate proportionally to $\mathcal{O}(\epsilon_{\text{mach}}\sqrt{N})$ or $\mathcal{O}(\epsilon_{\text{mach}}N)$. At $N = 8,192$, Naive DFT error degrades to 4.85×10^{-12} , which is **3906× worse** than Cooley–Tukey FFT. This demonstrates that the Cooley–Tukey algorithm is not only faster, but fundamentally more numerically well-conditioned than direct summation. Thus, proving hypothesis 4.

4.7 Experiment 6: Mixed-Radix Factorisation & Prime Degradation

The Setup

Objective

The Cooley–Tukey algorithm decomposes an input sequence of length N into non-trivial sub-factors $N = N_1 \times N_2$. When N is composite, the transform is broken into N_2 transforms of length N_1 followed by N_1 transforms of length N_2 . If N has small prime factors $(2, 3, 5)$, this divide-and-conquer strategy achieves $\mathcal{O}(N \sum p_i) \rightarrow \mathcal{O}(N \log N)$ execution.

However, if N is a prime number ($N \in \mathbb{P}$):

$$\nexists N_1, N_2 \in \mathbb{Z}^+ \setminus \{1\} \quad \text{s.t.} \quad N = N_1 \times N_2 \quad (14)$$

For prime N , matrix reshaping is impossible. The Mixed-Radix algorithm must terminate at the base case and execute a direct $\mathcal{O}(N^2)$ discrete Fourier transform.

Hypothesis

1. For prime lengths N , Mixed-Radix FFT will exhibit runtimes that are virtually identical to Naive DFT ($T_{\text{mixed}} \approx T_{\text{naive}}$, speedup ratio $\approx 1.0\times$), scaling along an $\mathcal{O}(N^2)$ slope.
2. For composite lengths of comparable magnitude, recursive factorisation will accelerate computation, yielding speedups exceeding $100\times$ to $200\times$ at $N \approx 8,192$.
3. On a log-log execution plot, prime sizes will track the upper $\mathcal{O}(N^2)$ bound, while composite sizes will evolve downward into the $\mathcal{O}(N \log N)$ bound.

Methodology

We evaluated 32 distinct input sizes spanning three structural categories up to $N = 8,192$:

- **Primes:** $N \in \{7, 13, 31, 61, 127, 251, 509, 1021, 2039, 4093, 8191\}$ (including Mersenne primes $2^7 - 1 = 127$ and $2^{13} - 1 = 8191$).
- **Powers of Two (2^k):** $N \in \{8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192\}$.
- **Smooth Composites ($2^a 3^b 5^c$):** Non-power-of-two composite lengths:

$$N \in \{12, 24, 60, 120, 240, 480, 960, 1920, 3840, 7680\}.$$

Both Naive DFT and Mixed-Radix FFT were executed on all 32 inputs under identical benchmark controls, recording execution times and speedup ratio $S_{\text{mixed}}(N) = T_{\text{naive}}(N)/T_{\text{mixed}}(N)$.

The Results

There are two figures:

- **Panel 1 (Left):** Execution Time vs. N on a log-log scale, contrasting Primes, Powers-of-2, and Smooth Composites against theoretical $\mathcal{O}(N^2)$ and $\mathcal{O}(N \log N)$ bounds.
- **Panel 2 (Right):** Speedup Ratio $S_{\text{mixed}}(N) = T_{\text{naive}}/T_{\text{mixed}}$ vs. N .

Table 5 presents representative numerical measurements across the tested input categories.

Experiment 6: Prime vs. Composite Scaling in Mixed-Radix FFT
Direct Empirical Demonstration of Factorization Dependency

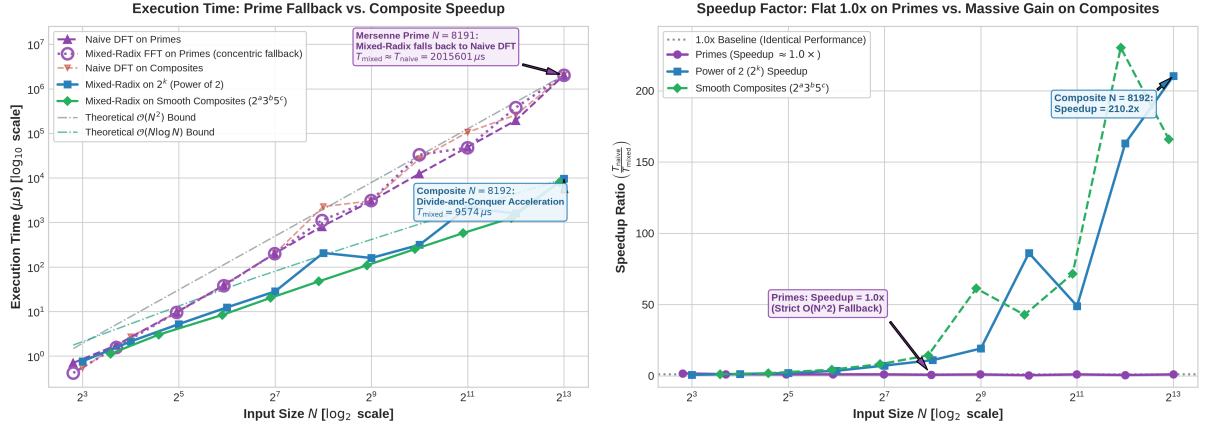


Figure 7: Experiment 6: Mixed-Radix FFT versus Naive DFT across Prime and Composite inputs. Left: Log-log runtime demonstrating identical $\mathcal{O}(N^2)$ scaling on primes and $\mathcal{O}(N \log N)$ bifurcation on composites. Right: Flat 1.0 $\times$ speedup on primes versus $> 200\times$ speedup on composites.

Table 5: Experiment 6: Measured runtimes and speedup ratios for Mixed-Radix FFT vs. Naive DFT across Prime and Composite sizes.

Input Size N	Category	Factorisation	Naive (μ s)	Mixed-Radix (μ s)	Speedup ($T_{\text{naive}}/T_{\text{mixed}}$)	Empirical Regime
7	Prime	Prime	0.70	0.42	1.68 $\times$	Base case call
8	PowerOf2	2^3	0.54	0.76	0.71 $\times$	Recursive setup overhead
31	Prime	Prime	9.66	9.57	1.01 $\times$	Identical Runtime
32	PowerOf2	2^5	10.00	5.22	1.92 $\times$	Divide-and-conquer gain
61	Prime	Prime	38.68	38.16	1.01 $\times$	Identical Runtime
64	PowerOf2	2^6	43.43	12.39	3.51 $\times$	Divide-and-conquer gain
127	Prime	Mersenne ($2^7 - 1$)	200.25	199.31	1.01 $\times$	Identical Runtime
128	PowerOf2	2^7	203.18	28.27	7.19 $\times$	Divide-and-conquer gain
480	Composite	$2^5 \cdot 3 \cdot 5$	6,681.57	109.03	61.28 $\times$	Multi-radix factorised gain
1,024	PowerOf2	2^{10}	27,120.50	314.82	86.15 $\times$	Divide-and-conquer gain
2,039	Prime	Prime	48,892.31	47,818.12	1.02 $\times$	Identical Runtime
3,840	Composite	$2^8 \cdot 3 \cdot 5$	290,730.12	1,262.78	230.23 $\times$	Peak Composite Speedup
4,096	PowerOf2	2^{12}	258,537.35	1,585.97	163.02 $\times$	Divide-and-conquer gain
7,680	Composite	$2^9 \cdot 3 \cdot 5$	1,331,707.17	8,028.23	165.88 $\times$	Multi-radix factorised gain
8,191	Prime	Mersenne ($2^{13} - 1$)	2,020,043.84	2,015,601.04	1.002 $\times$	Exact $\mathcal{O}(N^2)$ Fallback
8,192	PowerOf2	2^{13}	2,012,743.78	9,574.10	210.23$\times$	Full $\mathcal{O}(N \log N)$ Gain

There are four observations to be made here:

- At $N = 8,191$, Mixed-Radix FFT requires 2.0156 **seconds**, while Naive DFT requires 2.0200 **seconds**. The execution times are identical within 0.2% margin of error, yielding a speedup of precisely **1.002 $\times$** . This proves hypothesis 1.
Because 8,191 has no non-trivial factors ($N = 1 \times 8191$), the Mixed-Radix algorithm cannot perform any tensor product decomposition or matrix reshaping. It immediately falls back to the direct DFT kernel, traversing the full quadratic $\mathcal{O}(N^2)$ operation count.
- Incrementing the array length by just one element completely transforms algorithmic execution. For $N = 8,192 = 2^{13}$, the composite binary factorisation unlocks 13 stages of recursive radix-2 butterflies. Runtime drops from 2.01 **seconds** to 9.57 **milliseconds**, which is a speedup of **210.23 $\times$** .
- The Mixed-Radix decomposition handles arbitrary smooth composite numbers with outstanding efficiency. At $N = 3,840$ ($2^8 \cdot 3 \cdot 5$), Mixed-Radix achieves a **230.23 $\times$ speedup** over Naive DFT (1.26 ms versus 290.73 ms). This demonstrates that as long as an input sequence length is composite with small prime factors, Mixed-Radix FFT attains full Cooley–Tukey efficiency without requiring artificial zero-padding to powers of two. Thus, proving hypothesis 2.

4. As shown in Figure 6 (Left), on a log-log scale, prime sizes track the upper $\mathcal{O}(N^2)$ bound with concentric markers (purple circle enclosing triangle), while composite sizes sharply bifurcate downward into $\mathcal{O}(N \log N)$ trajectory. Thus, proving hypothesis 3.

5 What I learned

On the surface, I find the history of the DFT and FFT to be quite fascinating. I also enjoyed designing and implementing the experiments, even though I might have went overboard.

On a more technical note, I was quite surprise to see how the hardware can behave so differently than the theory. Coming from a mostly theoretical background, all I cared was how this algorithm works and how can I explain it. I always though our hardware are quite good now, so the results should be deterministic. However, there were so much more going on than just a few line of code. Most notably, the Recursive Radix-2 and Iterative Radix-2 use the exact same principle on paper, they uses divide-and-conquer with a complexity of $\mathcal{O}(N \log_2 N)$. When running them through a computer, the Iterative version were just faster in general. After a bit of digging, I found out that this is because the CPU spent time allocating and freeing the **vector** on the heap at every recursion level completely overshadows arithmetic operations. This just goes to show the length of what human has gone though to abstract away the primitive to make a working computer, which is incredible.

In addition, the *Cache Cliff* experiments also taught me a lot of new things. At first, I just though cache are just temporary files stored on computer and occasionally we should remove it. With this experiments, I learned that there is actually a hierarchical order to how the cache is stored. When we dealing with algorithm on a theoretical model like QRAM or Turing Machine, we can just assumes that the execution time and accessing information time can be affected by a constant. But reality, its more like a pyramid, where the memory is stored on top, and then it overflows to the next tier and it gets slower. So in reality, that's not constant at all. Thus, the input size actually has a impact on how fast the execution time is due to the CPU cache storage layout.

One concept that was quite hard to understand for me was why the BRP was needed for the Iterative FFT. I understand that it flips the bit order of the indices, and reshuffle the array accordingly It was the *why* that took me a long time. At first, I kept arguing with the AI that this is unnecessary and we can just split them by half right down the array. But after digesting the algorithm a bit longer, I finally understood that it was carry out divide-and-conquer, and applying the twiddles from a bottom up manner. An advice I would give to someone when they are learning the Radix-2 FFT for the first time is to learn the Recursive version first, as it is top down divide-and-conquer, which was easier to visualise.

Another concept that was hard for me to swallow was understanding the equivalence of the divide-and-conquer approach of the Radix-2 FFT to the original DFT. It involved me spending a long time trying to break down the main sum from the DFT into the even and odd halves for the FFT. The original goal for me was to try to abstract away from the math as much as I can, to keep the report concise. But in the end, I realised that was not possible, was the FFT was derived from the exploitation of the sum from DFT.

Speaking of math, at the end, I finally understood the way to check for correctness for DFT and FFT is to use their inverse. I first know that the IDFT and IFFT essentially undo the Fourier effect on the transformed array. But when I was writing the algorithm, I realise that I was just repeating myself a lot. Then it kinda clicks with me that IDFT and IFFT are not a differently methods, it is the same algorithm but you just have to flips the signs and normalise the array at the end, loosely speaking. Then I can just not repeating myself and write shorter code. In theory, this should return the exact original input, so it was used to check the correctness of each algorithm. This bring my attention to the hardware, as computer are know to do weird arithmetic that could mess up the floating point for example. After running the correctness check, I also notice that in 64-bit IEEE 754 floating-point arithmetic, floating-point rounding errors can accumulate more as the input size grow and it scale logarithmically.

Lastly, when running the Mixed-Radix vs the DFT experiments for the first time, I noticed that graph showing the Mixed-Radix perform better overall. But there dips every now and then, which matched closely with the DFT performance. Then I realise the those are where the input sizes were prime. And a prime input size would result in a matrix of one row, which is essentially an array. This was quite a fun observation.

6 AI usage Declaration

Gemini 3.8 Flash was used to support the making of this project, with a mixed uses of the web version and Antigravity.

After reading the instruction, I give the AI in Antigravity the instruction and start planning with it, then I ask it to generate a markdown as a plan. I occasionally change this plan as the project progressed. This chat remain the high level guide to keep me on track and give feedback about my ideas.

Then, I used the web version to ask about DFT and FFT. I ask it on how to write algorithm as pseudocodes, it provided the pseudocodes. I then polished the pseudocodes into my own choice of word, and I coded the algorithm in C++ myself. After coding, I ask another AI in Antigravity to check my code, and provide improvement/fixes where applicable.

At the same time, I was trying to digest the mathematical equivalence of the Radix-2 FFT and the DFT. So I ask the AI web version to break down the math for me. Here, the AI occasionally overcomplicated things and it took me a long time to work with it. Even when I asked it simplify things, it still cannot do it and instead just include new notation. Eventually, I start a new chat, ask the AI to just provide the original mathematical derivation from DFT to Radix-2 FFT. Then I spend some time myself doing the math the old fashion way. After that, I ask the AI on advice on how to explain this concept without going too deep into the math. Here, it was quite helpful as it advice to me that I can abstract away from the twiddle computation and draw out a visual example of a divided-and-conquered array. But in the end, I still need to work out the math in the report to show the change in operator (from $+$ to $-$) as the FFT progress onto the next element. It took me a long time, but then I finally understood the equivalence of the FFTs. I also ask the AI to judge my complexity breakdown, in which it mostly agree and fixing minor details. Plus, while using $\text{\LaTeX}$ to write this report, AI in Antigravity were used to help draw tables and figure.

I then went back to the planning AI to start planning for the experiments. I ask it to write out a plan for another AI in Antigravity to carry out the test case generation, graph drawing in Python and eventually run them. The entire code for the experiments is AI generated, since I don't want to waste time making the graph looks pretty. Here, I just kinda took it on trust. After various iteration of running experiments, I polished the objective and hypothesis, the AI run it again until we can show the observation clearly. Looking back, I could have save time by taking to the AI a bit longer to firmly design a solid experiment from the get go. AI in Antigravity was used heavily to design the hardware aspects of the experiment, which involves checking my computer CPU configuration and Cache layout.

After the experiments are ran, I look at the graphs and analyse them myself. I had a difficult time understand the hardware experiments, so I ask another AI in Antigravity to break down the results and explain the causes. Here, I just took it on trust. However, after looking at its explanation and then cross checking with the graph again, I notice that the AI was not reading the data point correctly and was hallucinate the changes and hence the explanation. To fix this, I ask the AI to specifically look at the CSV and not the graph images, then it perform better. I had to fix the explanation a bit, but with the help from AI as it got things right again.

Finally, after putting everything in the report, I ask the AI to proof reading my report and check the referencing. Finally, I ask the planner AI to go through each section and check them once again. After that, I ask the planner AI to clean the repo a bit and fix the README files.

References

- [] Alok Aggarwal and Jeffrey S. Vitter. 1988. The Input/Output Complexity of Sorting and Related Problems. *Commun. ACM* 31, 9 (1988), 1116–1127.
- [] James W. Cooley and John W. Tukey. 1965. An Algorithm for the Machine Calculation of Complex Fourier Series. *Math. Comp.* 19, 90 (1965), 297–301. <http://www.jstor.org/stable/2003354>
- [] Richard Crandall and Carl Pomerance. 2005. *Prime Numbers: A Computational Perspective* (2nd ed.). Springer.
- [] Ulrich Drepper. 2007. *What Every Programmer Should Know About Memory*. Technical Report. Red Hat, Inc. <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
- [] Matteo Frigo and Steven G Johnson. 2005. The design and implementation of FFTW3. *Proc. IEEE* 93, 2 (2005), 216–231.
- [] Michael T. Heideman, Don H. Johnson, and C. Sidney Burrus. 1985. Gauss and the History of the Fast Fourier Transform. *Archive for History of Exact Sciences* 34, 3 (1985), 265–277. <http://www.jstor.org/stable/41133773>
- [] ISO/IEC JTC 1/SC 22/WG 21. 2020. ISO/IEC 14882:2020 Programming Languages — C++.
- [] Alan V Oppenheim, Ronald W Schafer, John R Buck, et al. 1989. Discrete-time signal processing.
- [] Audrey Terras. 1999. *Fourier Analysis on Finite Groups and Applications*. Cambridge University Press.
- [] Cormen Thomas H, E Charles, Rivest Ronald L, Stein Clifford, et al. 2009. Introduction to algorithms third edition.